

To-Do List Manager - Final Proje Raporu

Ders: Bulut Mimarilerinde Test Mühendisliği

Proje Adı: To-Do List Manager (Görev Yöneticisi)

Tarih: 17 Mayıs 2026

Ad Soyad : Onur Burak Su

Numara : 170422505

1. Giriş

Bu projenin amacı, modern bulut mimarilerinde yazılım geliştirme, test mühendisliği ve dağıtım (deployment) süreçlerinin uçtan uca (end-to-end) nasıl kurgulandığını pratik bir uygulama üzerinden göstermektir. Proje kapsamında, kullanıcıların günlük görevlerini (task) oluşturabildiği, listeleyebildiği, tamamlayabildiği ve etiketleyebildiği (tag) modüler bir To-Do List Manager (Görev Yöneticisi) mini-uygulaması geliştirilmiştir.

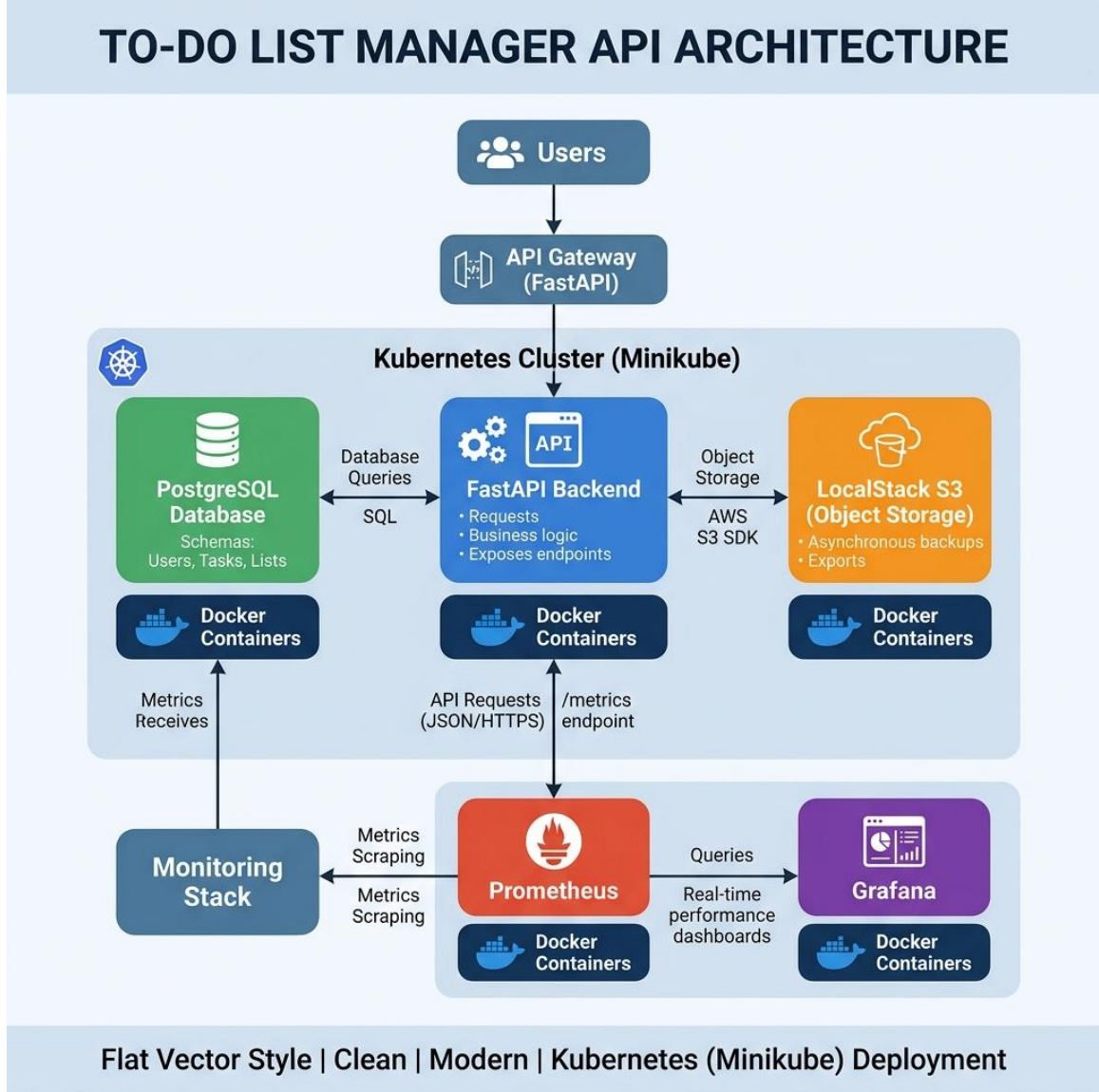
Bu uygulamanın seçilme nedeni, temel CRUD operasyonlarını içermesi, birbiriyle ilişkili veritabanı tablolarına sahip olması (Many-to-Many) ve aynı zamanda test, CI/CD, konteynerizasyon gibi karmaşık devops kavramlarını uygulamak için yeterince sade ama kapsayıcı bir alan sunmasıdır.

2. Mimari

Projemiz, modern mikroservis mimari standartlarına uygun olarak tasarlanmıştır.

- **Backend (Sunucu Katmanı):** Yüksek performanslı ve asenkron yapısı sebebiyle Python FastAPI framework'ü kullanılmıştır. Proje, routes (istek yönlendirme), services (iş mantığı) ve models (veritabanı şemaları) olmak üzere 3 katmanlı modüler bir mimariyle tasarlanmıştır.
- **Veritabanı Katmanı:** Geliştirme ortamında SQLite, test ve üretim ortamı simülasyonlarında ise PostgreSQL kullanılmıştır. Veritabanı işlemleri SQLAlchemy 2.0 ORM ile yönetilmektedir.
- **AWS / LocalStack Entegrasyonu:** Gerçek bir bulut ortamı simülasyonu için projeye LocalStack entegre edilmiştir. Uygulama, boto3 kütüphanesi aracılığıyla veritabanındaki görevleri JSON formatında sanal bir S3 Bucket'ına yedekleyebilmektedir.
- **Konteyner ve Kubernetes (K8s):** Uygulama, Multi-Stage bir Dockerfile ile konteynerize edilmiştir. Dağıtım aşamasında Minikube kullanılmıştır. Ayarlar ConfigMap üzerinden verilmiş, Deployment nesnesi ile pod'lar ayağa kaldırılmış ve NodePort Service ile dış dünyaya açılmıştır.
- **İzleme (Monitoring):** Uygulamaya prometheus-fastapi-instrumentator eklenmiş ve /metrics üzerinden Prometheus'a veri sağlanmıştır. Bu metrikler Grafana'da görselleştirilmektedir.

Aşağıda sistemin genel mimari diyagramı görülmektedir:



3. Test Stratejisi

Projede test piramidi (Test Pyramid) yaklaşımı benimsenmiştir. **Genel kod kapsamı (Coverage) hedefi %70 olarak belirlenmiş, ancak testler sonucunda %83 oranına ulaşılmıştır.**

1. **Birim Testler (Unit Tests):** pytest framework'ü kullanılarak services ve routes katmanları izole bir şekilde test edilmiştir. Test verileri Factory Boy ve Faker kullanılarak dinamik olarak üretilmiştir.
2. **Entegrasyon Testleri:** Veritabanı ORM yapısının doğruluğunu test etmek için Testcontainers kullanılarak Docker üzerinde geçici bir PostgreSQL veritabanı ayağa kaldırılmıştır.
3. **Arayüz (E2E) Testleri:** Microsoft Playwright framework'ü ile statik HTML arayüzündeki temel senaryolar otomatik tarayıcı üzerinde test edilmiştir.

4. Pipeline & Deploy

Sürekli Entegrasyon (CI/CD) süreçleri için GitHub Actions kullanılmıştır. Repoya yapılan her push işleminde .github/workflows/ci.yml devreye girmektedir.

GitHub Actions Akışı:

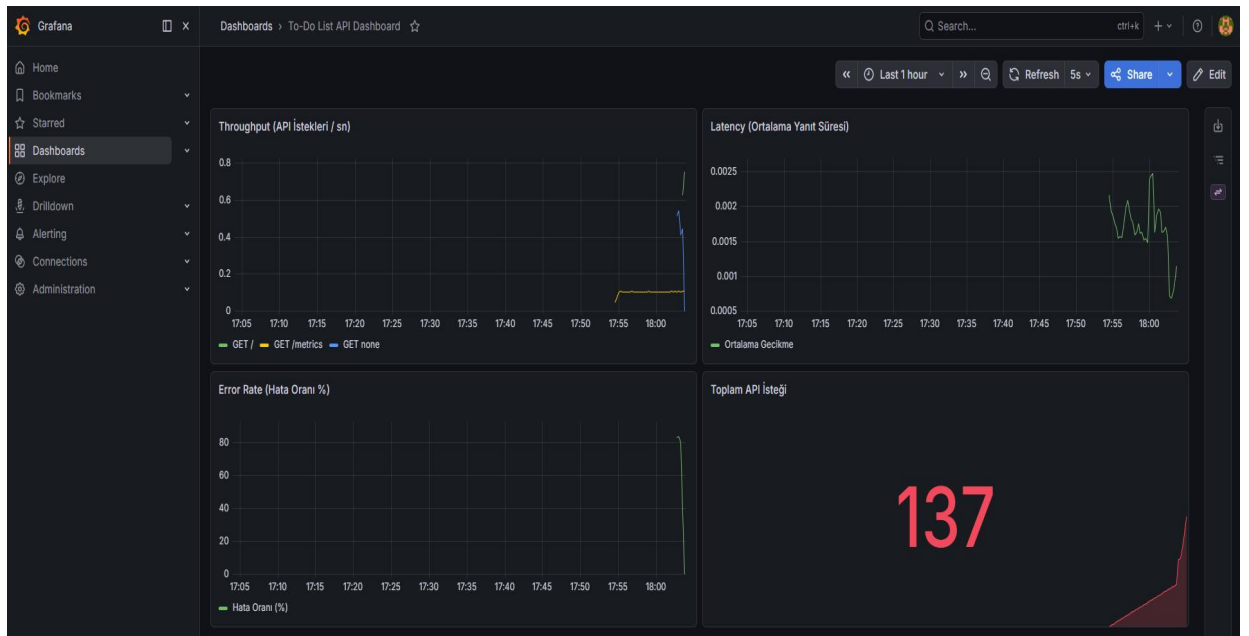
1. Linting: flake8 aracı ile syntax kontrolü.
2. Pytest: Tüm Unit ve E2E testlerinin koşulması.
3. Docker Build: İmajın hatasız oluşturulduğunun doğrulanması.
4. Deploy (Simülasyon): İmajın geçici bir container olarak çalıştırılması.
5. Smoke Test: Ayağa kalkan uygulamanın /health endpoint'ine istek atılarak kontrol edilmesi.

Deploy Stratejisi: Canlı ortam dağıtımı için Kubernetes (Minikube) manifestleri hazırlanmıştır. Uygulamanın çökmesi durumunda kendi kendini yenilemesi için livenessProbe ve readinessProbe yapılandırılmıştır.

5. Performans & Gözlemlenebilirlik

Sistemin yoğun yük altındaki dayanıklılığını ölçmek için k6 kullanılarak bir performans testi (load-test.js) yazılmıştır.

- **k6 Senaryosu:** Yük aşamalı olarak artırılmış; 10 saniyede 20 kullanıcıya çıkılmış, 30 saniye boyunca tepe yükü korunmuştur.
- **Hedefler (Thresholds):** İsteklerin %95'inin (p95 latency) 500ms'nin altında olması ve hata oranının %1'den düşük olması hedeflenmiştir.
- **Monitoring:** FastAPI üzerinden toplanan Prometheus metrikleri ile Grafana'da bir dashboard kurgulanmıştır.



6. Sonuç & Öğrendiklerim

Bu proje boyunca sıfırdan başlayan bir kod tabanının adım adım üretim standartlarına (production-ready) nasıl getirileceği pratik edilmiştir.

Sayılarla Özet:

- 10 farklı REST API endpoint'i
- %83 Test Kod Kapsamı (Coverage)
- 11 Postman Assertion Testi
- 5 Kademeli tam otomatik CI/CD Pipeline

Karşılaşılan Zorluklar ve Çözümleri:

- Test İzolasyonu: Testler arasında veritabanı durum sızıntısı yaşandı, conftest.py içerisinde veritabanının her fonksiyonda sıfırlanması sağlanarak çözüldü.
- Testcontainers ve Docker: Windows ortamlarında Docker erişim sorunları try-except bloğu ile aşıldı.
- Ortam Değişkenleri: Güvenlik riski taşıyan .env dosyası yerine K8s ortamında ConfigMap kullanımına geçildi.

İleride Yapılabilecekler:

Sisteme JWT tabanlı kimlik doğrulama eklenebilir, Frontend React ile baştan yazılabilir ve Kubernetes dağıtımları ArgoCD ile otomatikleştirilebilir.

7. Kaynaklar

1. FastAPI Resmi Dokümantasyonu: <https://fastapi.tiangolo.com/>
2. SQLAlchemy 2.0 Dokümantasyonu: <https://docs.sqlalchemy.org/>
3. Pytest Dokümantasyonu: <https://docs.pytest.org/>
4. Docker Multi-Stage Builds: <https://docs.docker.com/build/building/multi-stage/>
5. Kubernetes (Minikube) Belgeleri: <https://kubernetes.io/docs/tutorials/hello-minikube/>
6. Playwright Python Dokümantasyonu: <https://playwright.dev/python/>
7. K6 Performans Testi Kılavuzu: <https://k6.io/docs/>